

Trioron — Methods (§3)

3 Methods

Notation. n is the number of nodes in a layer; d is the layer’s incoming dimension; B is batch size; t indexes tasks in a curriculum.

3.1 The Trioron Node

A *trioron* is a single computational unit characterized by three coupled state variables, generalizing the standard artificial neuron:

- $w \in \mathbb{R}^d$ — incoming weights (a row of the layer’s weight matrix).
- $\lambda \in \mathbb{R}_{\geq 0}$ — per-node *plasticity coefficient*: a per-unit scalar that scales the unit’s contribution to the elastic-weight consolidation (EWC) penalty. Larger $\lambda \Rightarrow$ stiffer; smaller $\lambda \Rightarrow$ more plastic.
- $u \in \mathbb{R}$ — per-node *utility score*: a running estimate of the unit’s contribution to the network’s outputs. Used as the signal for pruning decisions (cells with persistently low u are candidates for removal).

A *trioron layer* aggregates n trioron nodes that share an incoming dimension. We hold the per-node state in vectors $\lambda, u \in \mathbb{R}^n$ and a weight matrix $W \in \mathbb{R}^{n \times d}$ where row i is w_i . The forward pass for input batch $X \in \mathbb{R}^{B \times d}$ is

$$H = \sigma(X (r \odot W)^\top + b), \quad (1)$$

where σ is the activation function, $b \in \mathbb{R}^n$ is the bias, and $r \in [0, 1]^n$ is the layer’s *routing scale* — a per-node multiplicative gain on incoming weights, defaulting to all ones. The routing scale is the substrate for routing-starvation consolidation (§??).

Each layer additionally maintains EWC state: anchor weights W_{anchor} and b_{anchor} snapshotted at task boundaries, and Fisher-information accumulators F_W, F_b updated as a running EMA of squared gradients. Per-node λ is derived from the row-mean of Fisher: $\lambda_i = \text{mean}_j F_{W,i,j}$.

Epigenetic analogue. w is the synaptic-strength channel (modulated by experience-dependent LTP/LTD); λ is a per-cell modifier on plasticity, with biological correlates including DNA-methylation status of plasticity-related genes (e.g., BDNF) and perineuronal-net maturation, both of which gate how readily a cell’s synapses can be modified; u is the cell-importance signal that biological systems approximate via activity-dependent neurotrophin release. The triplet (w, λ, u) captures *which connections are present, how rigid they are, and how much the cell matters* — three quantities that biological systems modulate independently.

3.2 Continual-Learning Substrate (EWC with Per-Node Plasticity)

Following Kirkpatrick et al. (2017), we apply elastic weight consolidation to protect prior-task knowledge. The penalty for a single neuron layer is

$$L_{\text{EWC}} = \sum_i \lambda_i \left[\sum_j (W_{ij} - W_{\text{anchor},ij})^2 + (b_i - b_{\text{anchor},i})^2 \right]. \quad (2)$$

The total loss is $L = L_{\text{task}} + \beta \cdot L_{\text{EWC}}$ for an EWC strength β set per-curriculum. We refresh λ_i from Fisher at the end of each task (after a batched re-estimation pass over recent task data), then snapshot $W_{\text{anchor}} \leftarrow W$ as the new anchor. This separation — estimate Fisher at task end, then anchor — follows the original EWC recipe and avoids the EMA-drift artifacts of online updating during training.

For longer curricula we optionally accumulate Fisher across tasks via the Online EWC variant (Schwarz et al. 2018):

$$F_t \leftarrow \gamma \cdot F_{t-1} + F_{\text{new}}, \quad \gamma \in [0, 1], \quad (3)$$

with $\gamma = 1$ reducing to vanilla EWC. We use Online EWC $\gamma = 0.95$ as a competitor baseline in §4.

A key modification: when an apoptosis spike fires (§??), each node’s *effective* stiffness is scaled by $\max(0, 1 - p_i)$, where p_i is the node’s apoptosis pulse:

$$\lambda_i^{\text{eff}} = \lambda_i \cdot \max(0, 1 - p_i). \quad (4)$$

This temporarily reduces a surviving cell’s EWC pinning when a neighbor has just died, allowing it to absorb the dead neighbor’s role. Mechanistically analogous to acute glial-derived neurotrophic factor release post-apoptosis, which transiently increases plasticity in adjacent cells.

3.3 Structural Plasticity

The architecture supports in-place growth and pruning of nodes during training, with cross-layer consistency.

Cellular division (grow_node). Adds one row to a layer’s weight matrix, extends the per-node state vectors λ, u, r by one entry, and (if a downstream layer exists) extends that layer’s incoming dimension by one column. The new node is initialized fully plastic ($\lambda_{\text{new}} = 0, u_{\text{new}} = 0, r_{\text{new}} = 1$) so it can adapt freely while existing nodes remain protected by EWC. The new row’s incoming weights are initialized along the top principal direction of the residual signal $D = f(X_a) - f(X_b)$ at the layer below — the direction of unmet representational variance, computed by SVD over a probe batch.

Cellular pruning (prune_node). Removes a node’s row, contracts the state vectors, and drops the corresponding input column on the next layer. By default the pruned node’s outgoing column is *redistributed* to its weight-cosine-nearest peer on the next layer before removal, preserving approximate input–output behavior of the layer.

Growth trigger. A node is grown only when three conditions are jointly satisfied over a window of W steps:

1. *Loss plateau.* The loss has not improved by more than $\varepsilon_{\text{loss}}$ relative to the prior window.

2. *Rank saturation.* The effective rank

$$r_{\text{eff}} = \exp\left(-\sum_k p_k \log p_k\right), \quad p_k = \sigma_k / \sum_k \sigma_k,$$

computed from the normalized singular values of the latent activation matrix, has approached the latent dimension ($\text{latent_dim} - r_{\text{eff}} < \varepsilon_{\text{rank}}$).

3. *Gradient stability.* The median gradient norm is bounded $g_{\min} \leq |\tilde{g}| \leq g_{\max}$ — the optimizer is not exploding or vanishing.

The conjunction of plateau-and-saturation-and-stability ensures that the network grows only when it has *actually run out of capacity for the current task*, not merely when training is slow.

Resource ceilings. A pre-flight check rejects a proposed division if it would exceed a per-curriculum byte budget *cap_bytes*. The cap can be lifted between phases (we do this in the extension experiment of §?? and §4.5). This implements *resource awareness* — the network knows when not to grow even if growth would help. Biologically analogous to metabolic limits on adult neurogenesis.

Epigenetic analogue. Growth corresponds to adult neurogenesis (which remains active in the dentate gyrus and a few other regions throughout life); the rank-saturation trigger reflects the biological observation that new neurons are recruited preferentially when existing circuits saturate. Pruning corresponds to developmental and ongoing synaptic pruning, with cosine-nearest-peer redistribution as a soft analog of compensatory hypertrophy.

3.4 The Frustration Multiplier

We introduce a per-task plateau counter that scales the task loss when the optimizer gets stuck. For each task, accumulate the loss in windows of length W_f . At each window boundary, compare this window’s mean loss to the prior window’s. If improvement is below ε_f , increment a stuck counter s . The multiplier is

$$m = \min(M_{\max}, 1 + g \cdot \max(0, s - \tau + 1)) \quad (5)$$

with hinge threshold τ , gain g , and ceiling $M_{\max}$. The multiplier is applied *only to the task loss*, not the EWC penalty:

$$L = m \cdot L_{\text{task}} + \beta \cdot L_{\text{EWC}}. \quad (6)$$

This is mechanically equivalent to focal-loss / hard-example-mining: amplify the gradient signal on examples the optimizer isn’t progressing on, without amplifying the regularizer.

Epigenetic analogue. The biological correlate is the hypothalamic–pituitary–adrenal (HPA) axis stress response and its downstream effects on synaptic plasticity. Sustained behavioral stress (failure to make progress on a task) elevates glucocorticoids, which modulate gene expression at plasticity-related loci through epigenetic mechanisms — DNA-methylation changes at NR3C1 and BDNF, miRNA-mediated post-transcriptional regulation of plasticity proteins. Most epigenetic modifications require a stress signal to engage; the frustration multiplier is the architecture’s stress proxy.

3.5 The Dreaming Phase

Between training tasks we run an offline *dreaming block*. Frustration is disabled inside the dream block (no environmental stress during sleep); EWC remains active so that consolidation pulls weights toward their anchored state. The dream block runs four sequential stages:

$$\text{replay} \rightarrow \text{compression (optional)} \rightarrow \text{purge} \rightarrow \text{archive}.$$

Replay is the load-bearing mechanism in the working configuration; the compression-action design space is documented as ablation arms in §??.

3.5.1 Replay

Replay rehearses past tasks under EWC. We support two replay sources:

- *Episodic exemplars (hippocampal buffer)*. When real-data storage is permitted we maintain a buffer of K samples per past task, sampled uniformly at task end. During replay the buffer is shuffled with the current task’s batch (1:1 by default) and the cross-entropy loss is evaluated on both. $K \in \{1, 10, 20, 50\}$ is the storage axis we sweep in §4.3. $K = 0$ routes replay through the storage-free pseudo-rehearsal pathway of §??.
- *Manifold pseudo-rehearsal (storage-free)*. Per-class statistics of L0 activations are stored at task end and used to synthesize replay samples; see §?? for the full mechanism. This is the headline rehearsal channel.

Conceptually analogous to sharp-wave ripples during slow-wave sleep, which replay recent and remote memories at compressed timescales.

3.5.2 Redundancy Detection (Activation Cosine)

When dream-time compression is enabled, we score each pair of nodes (i, j) within a layer for *functional* redundancy via centered-activation cosine over a probe batch $a_i, a_j \in \mathbb{R}^B$:

$$\rho_{ij} = \frac{\langle a_i - \bar{a}_i, a_j - \bar{a}_j \rangle}{\|a_i - \bar{a}_i\| \cdot \|a_j - \bar{a}_j\|}. \quad (7)$$

Pairs with $\rho_{ij} \geq \tau_{ac}$ are candidates for consolidation. We use the activation signal (not weight-cosine) because it captures functional similarity across the data distribution rather than identical input-direction tuning. An early diagnostic confirmed that hidden-layer weight-cosine (0.6–0.8) is high but not functional, while latent-space weight-cosine is near zero by construction of the growth rule (§??) — so the weight-cosine signal is misleading. Layer-1 activation cosine reaches 0.85–0.92 in practice, well above the 0.59–0.81 weight-cosine ceiling.

3.5.3 The sRNA Cap

Each compression pass is bounded by a per-layer event cap $N_{\max}$. We observe empirically (§4) that σ -distance from baselines degrades monotonically with event count, motivating small caps; $N_{\max} = 1$ allows replay to absorb each consolidation event before the next event drifts on top.

Epigenetic analogue. The cap is a direct analog of resource-limited small-RNA pools that gate sleep-cycle synaptic homeostasis. miRNAs involved in synaptic remodeling are produced and consumed in stoichiometric amounts during a single sleep cycle; the cellular machinery cannot perform arbitrarily many consolidation events per cycle even when many candidates are present. The “fewer events = better” empirical pattern reproduces the biological observation that few consolidation events per cycle are protective.

3.5.4 Compression-Action Design Space (Ablation Arms)

Three substrate-respecting mechanisms for resolving a detected redundant pair (i, j) were tested as dream-time compression actions. None of them σ -confidently beats the no-compression baseline at 6 seeds; we document them here as the ablation arms reported in §4 and to motivate the substrate-preserving emphasis of §??.

1. *Synaptic downscale* (substrate-preserving). The peer (kept side) absorbs the victim’s outgoing column on the next layer; the victim’s outgoing is zeroed; the victim’s row at layer L is *not* touched. Architecture and parameter count are unchanged. Fisher entries on the victim’s outgoing edge are reset so the dormant unit is available for re-recruitment in future tasks.
2. *Routing starvation* (asymmetric ramp). Each event multiplies the victim’s routing-scale entry by $\alpha < 1$; below floor η the scale latches to 0 permanently. Bias is untouched, so the victim continues producing a constant downstream signal until downstream layers learn to absorb the constant via gradient descent — the unit “dies slowly” rather than instantly. Reversible: each `compress()` pass also runs a regrow step on non-victim, non-latched units, multiplying their scale by $1/\alpha$ (capped at 1). The kept side (“primary”) is selected by older `task_of_origin`, with larger outgoing-norm as tiebreaker.
3. *Apoptosis spike* (full-latch handler). When a routing-starvation event causes scale to cross η , two coupled mechanisms fire: (a) *redistribution* — the dead cell’s outgoing column is transferred uniformly across all surviving non-latched peers, then zeroed; (b) *spike* — the surviving peers’ apoptosis-pulse p_k is raised to $p_{\text{init}} \in (0, 1]$. The pulse decays multiplicatively by δ per dream block. Through the λ^{eff} modification of §??, neighbors of a fresh death train with reduced EWC stiffness for several dream cycles, allowing them to absorb the dead cell’s role.

We report engagement statistics for these mechanisms in §4 as evidence that the sRNA cap (§??) is the load-bearing decision: at the activation-cosine threshold $\tau_{\text{ac}} = 0.92$, half the seeds fire zero consolidation events — a bimodal engagement pattern that across-seed averaging conceals.

3.5.5 Purge

After compression, we drop nodes whose utility u_i falls below a threshold τ_u . Reuses the structural pruning machinery of §??, including cross-layer fan-in cleanup and cosine-nearest-peer redistribution. Distinct from the in-training pruner (§??): this is a sleep-time structural sweep, not an event-clock check during waking training.

3.5.6 Pseudo-code of a Dream Block

```
for each task t in curriculum:
    train_one_task(t)           # waking
    consolidate_task(t)        # Fisher update + anchor
```

```

if dreaming_enabled:
    decay_apoptosis_pulses_by_delta
    replay(buffer = hippo or manifold,
           steps = K_r,
           ewc_active = True)
    compress(signal = activation,
             threshold = tau_ac,
             action = ACTION,
             max_events_per_layer = N_max)
    purge(threshold = tau_u)
    archive_block(...) # 3.7

```

3.6 Manifold Replay (Storage-Free Pseudo-Rehearsal)

The headline rehearsal mechanism is *manifold replay*: a trion-native pseudo-rehearsal scheme that summarizes each past class as a small set of statistics in the network’s first-hidden-layer (L0) representation rather than storing exemplars.

Storage. When a task t ends we collect L0 activations for each class c that appeared in the task and store the per-class mean $\mu_c \in \mathbb{R}^{L_0}$ and per-dimension standard deviation $\sigma_c \in \mathbb{R}^{L_0}$ (a diagonal-Gaussian summary). Storage per class is $2 \cdot L_0 \cdot 4 = 1024$ bytes at $L_0 = 128$. A 30-class run costs ≈ 30 KB total — independent of the dataset size and of the number of samples per task.

Replay. During the dream block we draw pseudo-samples for each past class c :

$$\tilde{z} = \mu_c + \sigma_c \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad (8)$$

then forward them through the *upper* layers via `forward_from_layer(z_tilde, start = 1)` to obtain logits. Cross-entropy loss on $(\tilde{z}, c)$ is added to the total loss. Replay is performed at the same EWC strength as waking training.

Why per-class statistics work where logit-inversion fails. Three single-seed inversion variants — DeepInversion-style code synthesis (one-shot, refresh-all, and diversity-regularized) — all peaked at 0.27 full-softmax accuracy on chained-15. Logit inversion locates class- c attractor *peaks*, not class- c manifold *samples*; the resulting points concentrate at adversarial extrema that do not reflect actual class structure. This is the same pathology that defeated earlier engram-replay variants (gradient-ascent on real-mean seeds; L1-feature distillation; chain combinations of engram with differential L1 storage). The (μ, σ) summary lives on the real L0 manifold by construction; inversion attractors do not. An ablation removing per-class σ (μ -only replay) loses ≈ 0.07 absolute on full-softmax at 30 classes (single-seed), confirming that per-class noise scale is load-bearing — roughly 12% of manifold replay’s advantage.

Why L0 is frozen. Manifold replay assumes that L0’s encoding is approximately stationary across the curriculum, so that statistics collected after task t remain valid at task $t + k$. We enforce this by freezing L0 at initialization to a fixed random Gaussian projection (weights drawn from $\mathcal{N}(0, 2/d)$ at init and never updated). A separation-probe diagnostic on the chained-15 curriculum gives a class-prototype discriminability ratio of 1.063 in the frozen L0 space — modest but non-trivial, with Fashion-MNIST contributing the lowest separability (1.04) as expected from its irreducible

within-domain confusions. The ratio rules out a single-prototype scheme (L0-space class means alone are not separable enough) and motivates the stored- (μ, σ) sketch as a manifold summary rather than a class-prototype classifier.

Epigenetic analogue. Per-class (μ, σ) is a compact “engram fingerprint” — an offline sketch of the data manifold rather than a stored sample. Functionally closer to consolidated memory traces (gist-level recall) than to episodic exemplar storage (which the hippocampal-buffer baseline of §?? implements explicitly).

3.7 The Dream Archive

Long curricula accumulate parameters that should not keep moving — rows whose Fisher has plateaued and whose contribution is stable across recent tasks. We expose these as a *dream archive*: a two-phase mechanism for locking and quantizing such rows.

Phase 1 — mid-curriculum row archive. After each `consolidate_task`, every row of every layer is scored along three axes: (a) λ percentile (top `archive_lam_top_percentile` of the layer), (b) gradient-magnitude ceiling (median $|g|$ over the recent window below `archive_grad_mag_floor`), and (c) apoptosis-pulse bound ($p \leq \text{archive_pulse_max}$ — never archive a row recently pulsed by an apoptosis event). A row that satisfies all three for `archive_streak_threshold` consecutive tasks is *archived*: a per-row gradient mask is set so subsequent backward passes leave the row at its anchor value. The mask is enforced via `mask_archived_grads_all()` between `loss.backward()` and `opt.step()` (sequenced after PackNet/HAT grad surgery on competitor arms). A per-layer cap `archive_max_per_layer` prevents any one layer from being fully archived in a single pass.

Phase 2 — end-of-curriculum int8 quantization. At end of training we snap each archived row to int8 with a per-row symmetric scale (8 bits/weight plus one fp32 scale per row). Active (non-archived) weights are rounded to BF16 precision. The Phase 2 path is implemented as a snapshot–quantize–re-evaluate–restore sequence in the bench so we can report storage and accuracy numbers from the same trained checkpoint; the deployment path applies the snap permanently.

Storage accounting. With BF16 as the deployment baseline, archive + int8 reduces network weight storage by $\approx 40\%$ across all arms ($213.7 \rightarrow 127.5$ KB on the headline `grown_capped_dream` arm; full table in §4.4). We report deployment storage in the BF16 baseline because that is the honest device-side accounting; an FP32 baseline overstates the dream-archive win.

Why ternary fails on a frozen-L0 substrate. Ternary quantization (2 bits/weight) on the frozen-Gaussian L0 projection costs 0.31 absolute full-softmax accuracy on the smoke test: random-Gaussian weights have no concentration of magnitude near zero, so 2-bit quantization throws away most of the routing structure that L1 and the head learned to depend on. Int8 ($\approx 1/127 \approx 0.8\%$ per-weight noise) preserves the routing intact. Unlocking ternary would require quantization-aware training during the dream phase, or restricting ternary to layers whose weights are concentrated at zero (a learned, pruned L1 + head) — both deferred to future work.

Epigenetic analogue. Phase 1 corresponds to terminal cell differentiation — a cell that has settled into its mature role no longer participates in plasticity. Phase 2 corresponds to compression of consolidated memory traces into low-bandwidth long-term storage, freeing high-bandwidth substrate for new learning.

3.8 Mixed-Precision Substrate

For deployment on resource-constrained devices the architecture supports a mixed-precision mode in which weight Parameters (W, b) are stored in narrow precision (BF16) while all consolidation buffers — $W_{\text{anchor}}, b_{\text{anchor}}, F_W, F_b, \lambda, u, r, p$ — remain in FP32. The forward pass auto-casts inputs to the weight dtype, preserving the callable interface; the EWC penalty and Fisher accumulation upcast cleanly across the boundary.

This separation is necessary: pure-narrow-precision training degrades by $\approx 40\%$ due to optimizer-state precision loss in Adam’s running moments, but inference latency and memory cost are dominated by the weights, which can safely run in BF16. The intended deployment pattern is *train in FP32 during sleep cycles, deploy in BF16 for always-on inference*.

3.9 The Ship-Wake-Extend Loop

Deployment is not a one-shot training run that yields a fixed artifact; it is an iterative loop in which the model is shipped, woken on new data, allowed to grow plastic substrate, and re-shipped. We instantiate this loop with a *consolidation-dream pass* that fires once at a chosen task boundary:

1. **consolidation_dream_pass** — full-coverage replay over all past tasks with archive-aware gradient masking (locks honored), plus one additional **archive_block** call to catch rows that just settled.
2. Optional permanent int8 snap of the archived rows (no restore step, since this is the shipping checkpoint).
3. Cap lift: $\text{current_cap_bytes} \leftarrow \text{extension_cap_bytes}$. The network is allowed to grow new plastic substrate against the new task budget.
4. The training loop continues over the new tasks with shared internal state — manifold buffer, EWC anchors, archived rows, frustration counters all persist across the boundary.

The result is a smaller plastic envelope in deployment than in training, with new growth allocated only against the residual. A device-conscience deployment cannot revisit the original training data: it has only what it has stored (the manifold buffer, the archived weights) and what arrives in the new data stream. The consolidation-dream pass turns shipping into a controlled handoff: the network “sleeps long” before going to market, locking in the rows it will never need to move again, then wakes on new data in the field.

3.10 Multi-Branch Absorption

The ship-wake-extend loop (§??) lets a single organism continue learning across deployment cycles. A natural next question is whether two organisms trained *independently* on disjoint task streams can be *composed* into a single recipient without retraining either of them. We construct a multi-branch organism that does exactly this, with a shared frozen L0 random projection as the only structural prerequisite.

Architecture. A multi-branch organism comprises a shared frozen L0 (the same Kaiming-initialized random projection used in single-branch trioron, §??) and N parallel *branches*. Each branch is a self-contained donor produced by an independent training run: a frozen L1 substrate, a frozen head slice over its own covered class subset, and the per-class manifold archive $\{(\mu_c, \Sigma_c)\}_{c \in C_b}$ that is already computed for manifold replay (§??). The branches share the same L0 random

projection by being born at the same seed; this is the *shared-seed invariant*, and is what makes donor weights semantically valid in recipient code-space without distillation. We enforce non-overlap of class coverage across branches: $C_b \cap C_{b'} = \emptyset$ for $b \neq b'$.

Forward pass. For input x :

1. Project to code space: $z = \sigma(W_{L_0}x + b_{L_0})$ using the shared L0.
2. For each branch b , compute the per-branch head logits $\ell^{(b)} = \text{branch}_b(z)$ over C_b .
3. Compute per-branch generative log-likelihoods from the archive as a uniform mixture of per-class diagonal Gaussians:

$$\log p_b(z) = \log \left(\frac{1}{|C_b|} \sum_{c \in C_b} \mathcal{N}(z; \mu_c, \Sigma_c) \right).$$

4. Compute branch gates by softmax over branches:

$$g_b(z) = \frac{\exp(\log p_b(z)/T)}{\sum_{b'} \exp(\log p_{b'}(z)/T)},$$

where T is a temperature ($T = 1$ default). Rocky’s bet that bleed adds new function rather than destroys it is licensed here: multiple branches can co-fire.

5. Compose into combined logits over the union $C_\cup = \bigsqcup_b C_b$ via the principled mixture-of-experts log-probability:

$$\text{combined}_c(z) = \log g_{b(c)}(z) + \log \text{softmax}_c(\ell^{(b(c))}),$$

where $b(c)$ is the unique branch covering c . Slots not covered by any branch are not in $C_\cup$.

Why per-branch log-softmax. Each donor’s head was trained under its own active-class mask, so absolute logit *magnitudes* across donors are not comparable. Combining raw logits weighted by gates produces a biased argmax at full-softmax. Passing each branch’s logits through a log-softmax restricted to its own coverage normalizes the contributions to a common log-probability scale, after which the mixture combine is well-posed. This is an inference-only fix; no parameters are added.

Why the mechanism is zero-shot. Donor weights are byte-valid in the recipient because L0 is shared (matched random projection $\Rightarrow$ matched code distribution), and donor logits are placed into recipient class-namespaces via per-class indexing. The gate is computed analytically from the archive — a generative *Expert-Gate*-style router (Aljundi et al. 2017) using the manifold buffer trion already produces. No router training, no fusion-layer learning, no calibration pass; pure paste-and-go.

Optional dream-cycle calibration. For deployments where the full-softmax metric is load-bearing, a tiny additive bias offset can be trained to compensate for residual cross-donor magnitude mismatch. We sample synthetic codes $z \sim \mathcal{N}(\mu_c, \Sigma_c)$ from the union archive, forward through the assembled organism, and apply cross-entropy on the global label. Two variants:

- *per-branch* (N scalars): one bias broadcast uniformly over each donor’s covered slots. Mathematically task-aware-safe (a uniform per-donor shift is monotonic on within-task argmax because active classes within a chained-15 task always belong to a single donor by disjoint coverage).

- *per-class* ($|C_U|$ scalars): one bias per union class. Higher capacity, but *can* perturb within-task argmax; trades a small task-aware loss for a larger full-softmax gain.

The dream cycle uses only the donors’ own archives — no real-data retraining is performed.

3.11 Experiment Design

Curriculum (chained-15). A 30-class class-incremental benchmark drawn from three datasets in sequence:

- 10 MNIST digit tasks (one new digit class per task);
- 10 Fashion-MNIST tasks (one new garment class per task);
- 10 EMNIST-letters tasks A..J (one new letter class per task).

Each task introduces exactly one new class. At evaluation time the network is asked to classify across all classes seen so far (full-softmax) or restricted to the same task or domain (task-aware, domain-aware). The class-incremental framing forces the network to commit to absolute class identities — there is no task ID at inference — and is the standard hard regime for continual-learning evaluation.

Why class-incremental, not task-incremental. Task-incremental benchmarks supply the task ID at inference, which collapses the problem to learning a per-task classifier. Class-incremental does not, and is therefore the regime where partition-style methods (PackNet, HAT) suffer their characteristic full-softmax collapse. Reporting class-incremental from the start avoids confusing the comparison.

Extension curriculum (chained-23). chained-15 above, plus 8 additional EMNIST-letters tasks K..R introduced at the extension boundary; total 23 class-incremental tasks across 38 classes. Used only in the extension experiment of §4.5.

Network configuration.

- Three layers: input $\rightarrow$ L0 $\rightarrow$ L1 $\rightarrow$ head.
- L0: 784 $\rightarrow$ 128, frozen-Gaussian random projection (weights drawn from $\mathcal{N}(0, 2/784)$ at init and never updated). See §?? for why L0 is frozen.
- L1: 128 $\rightarrow$ 32 initial, grown adaptively up to $cap_bytes = 8\text{ K}$ trainable parameters during chained-15, lifted to 16 K at the extension boundary.
- Head: $L1_width \rightarrow n_{classes_so_far}$, grown adaptively as new classes appear.
- Activations: ReLU on L1, identity on the head.
- Optimizer: Adam, $lr = 3 \times 10^{-3}$.
- Epochs: 4 per task. Loss: cross-entropy on the active label set.
- EWC strength: $\beta = 1000$ during waking and dream-replay (matched).

Trioron arms.

- *fixed_ewc_small* — no growth; L1 fixed at the chained-15 cap. Isolates the EWC + manifold-replay contribution at matched parameters.
- *grown_capped_no_dream* — growth under EWC, no dream-time consolidation. Isolates the dream contribution.
- *grown_capped_dream* — growth + dream within cap. The default deployment arm.
- *grown_uncapped_dream* — growth past cap with manifold replay. The upper-bound headline arm.

Competitor families (5). We benchmark against one method from each major continual-learning family:

- *Hippocampal buffer (exemplar rehearsal)*. K samples per past class stored. $K \in \{1, 10, 20, 50\}$.
- *PackNet* (Mallya & Lazebnik 2018). Iterative magnitude pruning with per-task masks. *PackNet matched* matches our trainable-parameter budget; *PackNet standard* uses the original recipe.
- *HAT* (Serra et al. 2018). Hard binary attention masks per task, gating each layer. *Matched* and *standard* variants as above.
- *Online EWC* (Schwarz et al. 2018). Fisher EMA across tasks with $\gamma = 0.95$; learning-rate and β tuned via grid search on the no-dream loss.
- *LwF + EWC* (Li & Hoiem 2018; Kirkpatrick 2017). Logit-distillation toward the anchored network on old-class columns, in addition to the EWC penalty.

Metrics.

- *Full-softmax accuracy* (full): mean across all classes seen so far of top-1 accuracy with the unrestricted softmax. The hard class-incremental metric.
- *Domain-aware accuracy* (domain): top-1 accuracy restricted to classes that share a dataset domain (digits, fashion, letters) with the query. Measures preservation of coarse-grained discrimination.
- *Task-aware accuracy* (task): top-1 accuracy restricted to the classes of the *task* the query belongs to. The pairwise-discrimination metric; the right one for trioron’s deployment role of context-gated personalization atop a frozen LLM.
- *Forgetting* (forget): mean across past tasks of (final-task accuracy on that task) – (accuracy at the time the task was first introduced); positive values indicate forgetting, negative values indicate *negative forgetting* (accuracy improves over time after the task ended).
- *σ -vs-baseline*: paired-by-seed standardized effect size across $N = 3$ random seeds, reported alongside absolute Δ .

Storage accounting. We report deployment storage in two columns: BF16 baseline (the honest device-side number, since BF16 is the inference dtype) and FP32 baseline (the training-time number). The manifold buffer is counted at FP32 (per-class μ and σ , $2 \cdot L_0 \cdot 4 = 1024$ bytes/class). All headline storage numbers in §4 use the BF16 baseline.

Reproducibility. All benches run on commodity CPU. Each seed is fully deterministic given the seed value, modulo a known torch nondeterminism in archive-trigger timing (see §4.4 for the v1/v2 reconciliation). Launcher scripts under `experiments/` accept explicit seed lists; bench logs and CSVs from every reported run are committed to the public repository.